

Aakaar

A framework for governed Digital Workers — PRD v1 · Aarya (BizOps reconciliation) as the canonical first tenant

What Aakaar is. A multi-tenant SaaS framework that lets enterprises deploy autonomous Digital Workers — non-human roles that perform routine operational work continuously — with charter-bound mandates, four-tier decision rights, native maker-checker approvals, anomaly guards, evidence preservation, and a kill-switch. The LLM in Aakaar plans and explains; deterministic tools execute; named humans approve everything sensitive.

Aarya, the first tenant. A Digital Operations Analyst for Settlements & Reconciliation across UPI, Bharat Connect, and Banking Connect. She runs 24x7 against settlement windows; auto-resolves allowlisted matches; drafts (but never executes) sensitive reversals/refunds/disputes; creates ServiceNow break records; and routes every customer-impacting action to a named BizOps checker. Her charter v1.2 defines her mandate; Aakaar enforces it.

Why this is a framework, not a workflow tool

The previous PRD (v2) treated tenants as configuration. Reading Aarya’s charter makes clear that what an enterprise actually deploys is a *governed worker* — an entity in HCM, with a federated identity, a named accountable owner, scoped permissions, decision rights, an approval circle, and a charter that legal, compliance, audit, and architecture have signed. Workflows are the work; the worker is the artifact.

Aakaar therefore promotes Digital Worker, Charter, Decision Rights, and Maker-Checker to first-class concepts. The capability registry, agentic planner, Temporal orchestration, Kafka bus, and worker agents from v2 all remain — they are the foundation — but they are now framed by a governance layer above them and a quality-gate layer beneath them.

Out of scope for v1

- Tenant-authored custom adapters (deferred to v2 — the marketplace).
- Cross-worker workflow composition (one worker calling another’s actions).
- On-prem control-plane installation (v1 is SaaS-only; only the agent runs at the customer site).
- Federated multi-region active-active (v1 ships single-region active with warm DR; Aarya operates in a single India region).
- Direct customer communications and regulatory submission — permanently out of scope by design (consistent with Aarya’s charter section 6).

1. Glossary & knowledge base

These terms are precise. Where Aakaar’s framework terminology and Aarya’s charter language differ, both are listed and reconciled.

Term	Definition
Aakaar	The framework. Sanskrit / Hindi <i>aakaar</i> — “form, shape, mold.” The mold from which governed Digital Workers are cast.
Digital worker	A first-class entity in Aakaar representing an autonomous role (e.g. Aarya). Has identity, charter, scope, decision rights, schedule, owner, and lifecycle. Provisioned in the tenant’s HCM as a non-human worker.

Term	Definition
Charter	The signed mandate that bounds a Digital Worker. Versioned, peer-reviewed, governance-approved. Lists in-mandate vs. out-of-mandate actions, decision rights tiers, allowlists, thresholds, and escalation paths. The charter is the contract; the framework is the enforcement.
Decision rights	The four-tier classification applied to every proposed action: auto-execute · maker-checker · senior checker · hard stop . Tier is determined by the charter, the action category, the amount/impact, and the confidence.
Maker / checker	The maker (Aarya) prepares an action with full evidence. The checker (a named human) approves or rejects via an approval channel. Sensitive actions require maker-checker; high-impact actions require a senior checker.
Hard stop	An action class the worker cannot execute under any condition. Logged, ticketed, and assigned to a human owner. Examples: customer comms, regulatory submission, GL/customer-account posting, access changes.
Capability	A registered unit of work the framework can perform (e.g. <i>recon.match</i> , <i>sn.create_break</i> , <i>http.fetch</i>). Versioned, signed, sandboxed.
Capability registry	Authoritative catalogue of capabilities. Read by the planner, checked by the validator, dispatched by the orchestrator.
Schema registry	Authoritative catalogue of <i>data</i> schemas (NPCI files, NBBL formats, Banking Connect schemas, ServiceNow record shapes). Versioned independently of capabilities; referenced by ingestion.
Prompt registry	Authoritative catalogue of LLM prompts and narrative templates. Each prompt is versioned, peer-reviewed, and must pass replay/A-B testing before promotion.
Confidence score	A scalar attached to every proposed action by the planner and downstream tools. Charter rules cite thresholds (e.g. "auto-execute only above 0.95"). Below threshold → tier escalates.
Volatility guard	Anomaly gate. A 3σ spike or absolute threshold breach in a customer-impacting category triggers a hard stop and on-shift officer alert.
Velocity check	Anomaly gate. Prevents duplicate proposals (reversal, refund, compensation) for the same RRN/transaction reference within a configured window.
Grounding verifier	Quality gate for LLM output. Every transaction-specific fact in a narrative must trace to the deterministic evidence pack or approved static reference data; otherwise the narrative is rejected.
Evidence pack	The bundle attached to every action: source records, schema version, config version, correlation ID, intermediate decisions, confidence, and policy reasons. Persisted before any side effect; used for audit replay.
Correlation ID	A workflow-scoped identifier propagated across charter, plan, decision, approval, execution, and audit. Every external system (ServiceNow, Salesforce, target system, SIEM) records it.
Kill-switch	Single operational control owned by the charter's accountable owner and IAM. Suspends the Digital Worker, revokes service identities and tokens, freezes in-flight workflows.
Break-glass	The manual SOP a tenant invokes when Aakaar (or a connector) can't proceed. Aakaar logs the invocation and continues to track TAT and evidence; it does not gate the human SOP.
TAT	Turn-around time — the regulatory or contractual deadline a transaction or break must be resolved within. Aakaar exposes a TAT tracker that influences worker scheduling.
JIT elevation	Just-in-time, action-scoped permission elevation. Vended for the duration of a single execution ($TTL \leq 15$ min) and revoked automatically. Approval is captured in audit.

Term	Definition
iSAC	The Bank's identity service (Aarya's tenant context). Federated workload identities live here; Aakaar's identity broker integrates with it.
NPCI / NBBL	National Payments Corporation of India (UPI, settlement) and NPCI Bharat BillPay Ltd (Bharat Connect). Aarya's upstream payment networks.
Tenant	The enterprise customer that hosts one or more Digital Workers. Aarya's tenant is "the Bank."

2. Business PRD

2.1 Problem statement

Enterprises increasingly want to staff routine operational roles with autonomous software workers — reconciliation analysts, settlement operators, dispute drafters, KYC reviewers, monitoring analysts. Existing RPA platforms automate keystrokes; existing workflow engines execute DAGs; neither addresses the governance question that audit, compliance, and risk ask first: *who is accountable, what is this thing allowed to do, who approves what it does, how do we stop it, and how do we replay any decision?*

Aakaar exists to make Digital Workers safely deployable at enterprise scale. Aarya's charter, as written, can only be honored by a platform that treats charter, decision rights, and maker-checker as primitives — not as policies stapled to a generic workflow engine.

2.2 Target personas

Persona	Goals	What they need from Aakaar
Charter owner (e.g. Head of BizOps)	Deploy a Digital Worker with confidence; demonstrate to the board that risk is bounded.	Charter authoring + sign-off; live KPI/KRI scorecard; instant kill-switch; quarterly evidence pack.
Day-to-day owner (e.g. VP Ops Recon)	Hit settlement-window SLA; maintain match rate; reduce break backlog.	TAT-aware scheduling; checker dashboards; escalation paths; capacity surge handling.
Checker / senior checker	Approve quickly without losing context; reject with structured reasons.	Approval cards with full evidence + TAT countdown; deep links to source systems; batch review for low-risk.
Compliance / audit	Replay any decision; verify mandate adherence; meet regulator expectations.	Hash-chained audit; correlation IDs; replay sandbox; access recertification evidence.
Group CISO / IAM	Bound the worker's blast radius; prove identity and access controls.	Federated non-human identity; JIT elevation; per-tenant CMEK; sandbox; egress allowlists.
Platform engineer / architect	Add new capabilities without rebuilding governance every time.	Versioned capability/schema/prompt registries; replay test harness; OPA policies.

2.3 Value proposition

- **Charter is the contract.** Mandate, decision rights, allowlists, and thresholds live as versioned artifacts — not configuration drift across scripts.
- **Maker-checker by default.** Every sensitive action passes through a named human approver before any external side effect.
- **Defensible LLM use.** The model plans and explains; deterministic tools execute. The grounding verifier rejects narratives whose facts aren't in the evidence pack.
- **Zero silent failures.** Schema drift, low confidence, anomaly trips, missing approvers — all fail closed with structured reasons.
- **Replayable everything.** Charter version, prompt version, registry digest, evidence pack, and decisions are persisted; any sample action can be reconstructed bit-exact.
- **One kill-switch.** The charter's accountable owner can suspend the worker globally; service identities and JIT credentials are revoked in seconds.

2.4 Success metrics (12-month targets, framework level)

Metric	Target	Measurement
Charter-bound workers in production	≥ 5 across ≥ 3 customers	Worker registry
Auto-resolution rate (allowlisted, eligible)	30% → 70% over Year 1	Per-tenant scorecard
Step success after retries	≥ 99.5%	Orchestrator metrics
Audit replay success (sampled)	100%	Quarterly audit
Volatility / velocity hard-stop false-positive rate	≤ 2% of trips	Tuning telemetry
Checker median approval latency (within SLA)	≥ 95% of cases	Maker-checker metrics
Cross-tenant data incidents	0 (hard requirement)	SIEM + audit
Kill-switch RTO	≤ 60 s end-to-end	Tabletop & live drills

2.5 Aarya-specific outcomes Aakaar must enable

Drawn directly from Aarya’s charter (sections 8 and 12.7). Aakaar must make these measurable from day one:

Aarya KPI/KRI	Threshold (per charter)	Aakaar component supplying evidence
Recon match rate	≥ 99.5% auto-matched where eligible	Decision rights engine + capability metrics
RBI TAT breach count	0 avoidable breaches	TAT tracker + scheduler
Approval latency p95	Within Bank-defined settlement SLA	Maker-checker service
Schema-drift incidents	All fail-closed; no silent mis-parse	Schema registry + ingestion
Unauthorized action attempts	0 successful; all blocked & investigated	Decision rights + policy + audit
Audit replay success	100% for sampled actions	Evidence store + replay sandbox
Failover readiness (DR drill)	Pass quarterly within RTO/RPO	Hot-standby + state recovery
Prompt release quality	Zero critical hallucination defects	Prompt registry + grounding verifier
Rejection reason-code completeness	≥ 95% structured	Maker-checker rejection taxonomy

3. Digital Worker model & Charter

3.1 The Digital Worker entity

A Digital Worker is the unit of deployment in Aakaar. It bundles identity, charter, decision rights, schedule, capabilities, schemas, prompts, checker pools, and accountable owners into a single governed artifact. Aarya is one Digital Worker. The same tenant could later host additional workers (e.g. a KYC-review worker, a Cards-dispute drafter), each with its own charter and isolation.

Required attributes

Attribute	Purpose	Aarya example
Name	Display + audit reference	Aarya
Role	Functional title	Digital Operations Analyst — Settlements & Reconciliation
Department / domain	Tenant grouping	BizOps
Federated identity	Workload SVID + linked service identities	iSAC + ServiceNow + Salesforce + ...
Accountable owner	Charter sign-off + escalation	Head of BizOps
Day-to-day owner	Operational accountability	VP Operations — Recon
Working model	Schedule, on-call pairing	24x7 to settlement windows
Charter ref	Mandate version in force	aarya-charter@v1.2
Capability allowlist	What the worker may invoke	recon.* · sn.create_break · ...
Schema bindings	Data formats it ingests	npci.upi.v3 · nbbl.bbbs.v2 · ...
Prompt bindings	LLM prompt versions	triage@v4 · narrative@v7
Checker pool(s)	Named approvers per tier	BizOps shift checkers; senior checkers
Kill-switch owners	Who can suspend	Head of BizOps + IAM emergency process

3.2 Charter as a first-class artifact

A charter is a versioned, governance-approved document. Aakaar stores it structured (so policy and decision rights are machine-evaluable) *and* as the human-readable narrative the signatories actually approved. Both forms are signed and immutable; subsequent versions are a new charter, not a mutation.

Lifecycle

- **Draft** — Authored in Aakaar by the worker's owner; reviewers comment.
- **Review** — Routed to required reviewers (per Aarya: BizOps, Group CISO, Compliance, IS Audit, Group Architecture, DPO, IAM, ServiceNow Platform, Salesforce Platform).
- **Sign-off** — Required signatories approve; their digital signatures are bound to the document hash.
- **Active** — Worker is deployed; this charter version governs every action until amended.
- **Amendment** — Any material change to scope, access, model behavior, decision rights, or data handling requires a new charter version through the same review.
- **Retired** — Worker is decommissioned; charter is preserved for audit indefinitely.

3.3 Mandate & non-mandate

The mandate is expressed as two allowlists — in-scope action categories and out-of-scope hard stops — plus parametric thresholds. Aakaar's Decision Rights Engine evaluates every proposed action against this set; anything not explicitly in-mandate is treated as out-of-mandate by default.

Aarya example. *In-mandate: scheduled NPCI/NBBL pulls, 3-way recon, auto-match for allowlisted high-confidence non-customer-impacting categories, ServiceNow break creation, maker-only drafting of reversal/refund/dispute, internal status emails to approved BizOps lists. Out-of-mandate: customer communication, GL/customer-account posting without checker, regulatory submission, ownership/permission changes, anything outside the BizOps reconciliation perimeter.*

3.4 What the framework enforces vs. what it observes

Some charter clauses are machine-enforceable (allowlist of actions, confidence thresholds, recipient allowlists, JIT credential TTL). Others are observational — the framework records evidence but cannot *prevent* the breach (e.g. a checker approving without reading the evidence pack). For those, Aakaar provides metrics and replay so governance review can detect drift.

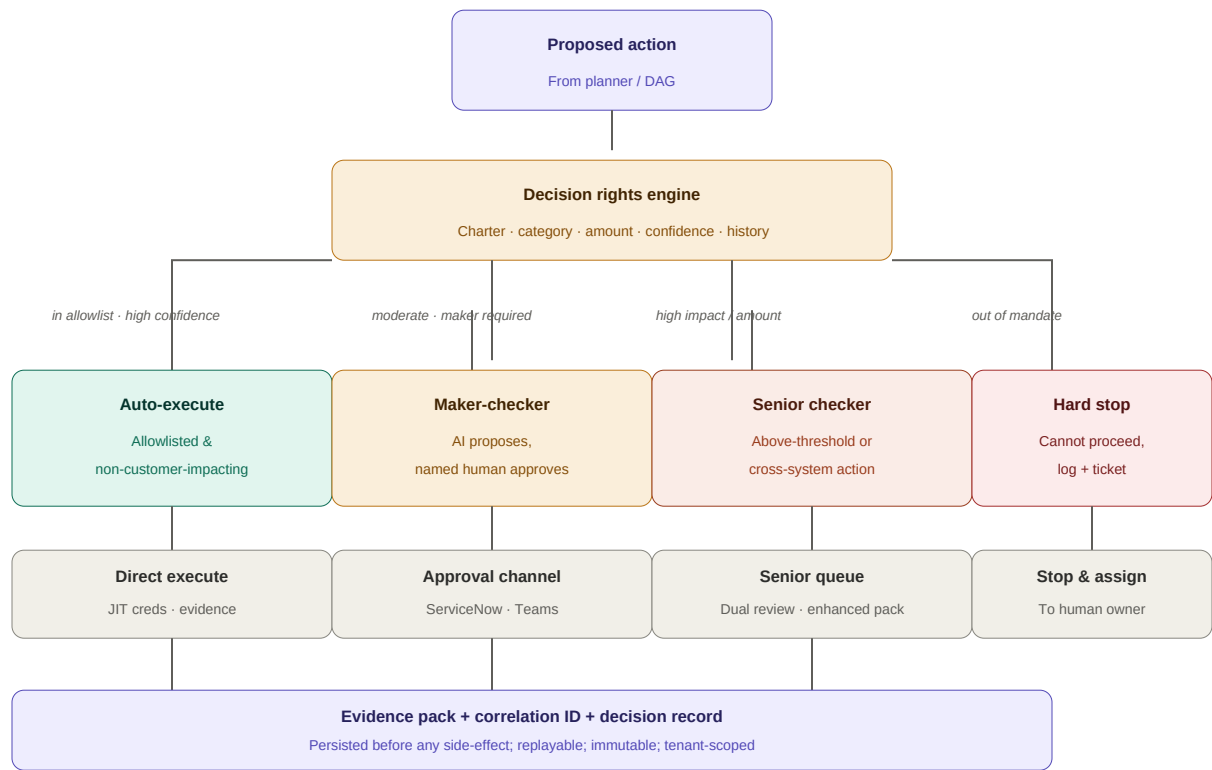
4. Decision Rights framework

Every proposed action passes through the Decision Rights Engine before any external side effect. The engine is deterministic and charter-driven; the LLM's confidence is one input, never the only one.

4.1 The four tiers

Tier	When it applies	Aakaar behavior	Aarya example
Auto-execute	Action is in mandate, in the auto-allowlist, confidence above threshold, no anomaly trip.	Vend JIT creds, execute via adapter, persist evidence, emit event.	Exact recon match; in-tolerance timing diff; ServiceNow break creation.
Maker-checker	Action is sensitive but routine; needs a named human approver from a designated pool.	Aarya prepares evidence pack; open approval ticket via the channel; await named approver.	Failed-debit reversal preparation; refund credit preparation; Salesforce Case status update.
Senior checker	Above-threshold amount, cross-system corrective action, high-value Banking Connect cases, dual-review categories.	Same as maker-checker but routes to senior pool, with an enhanced evidence pack (extended history).	Above-threshold reversals/compensation; duplicate debit; cross-system corrections.
Hard stop	Out-of-mandate, unsupported schema, confidence below threshold, anomaly trip, no approver available, kill-switch active.	Action is refused. A ServiceNow ticket is created and assigned. Reason is logged in structured form. Worker proceeds to next item.	Customer comms; direct GL/customer-account posting; regulatory return; access change; schema-drift event.

4.2 Routing diagram



Every proposal is classified by the engine before any external side effect. Tier is a function of the charter, not the LLM.

Diagram — the four tiers, the conditions that select each, and the evidence pack persisted on every path.

4.3 Inputs to the engine

- **Action type** (the capability invocation).
- **Action parameters** (amount, category, customer impact flag, recipient).
- **Charter version** currently in force for the worker.
- **Confidence score** from the planner / classifier.
- **History** — recent identical or near-identical proposals (velocity).
- **Anomaly state** — volatility guard tripped or quiet.
- **Approver availability** — required pool has live members.
- **Worker state** — not suspended, kill-switch off.

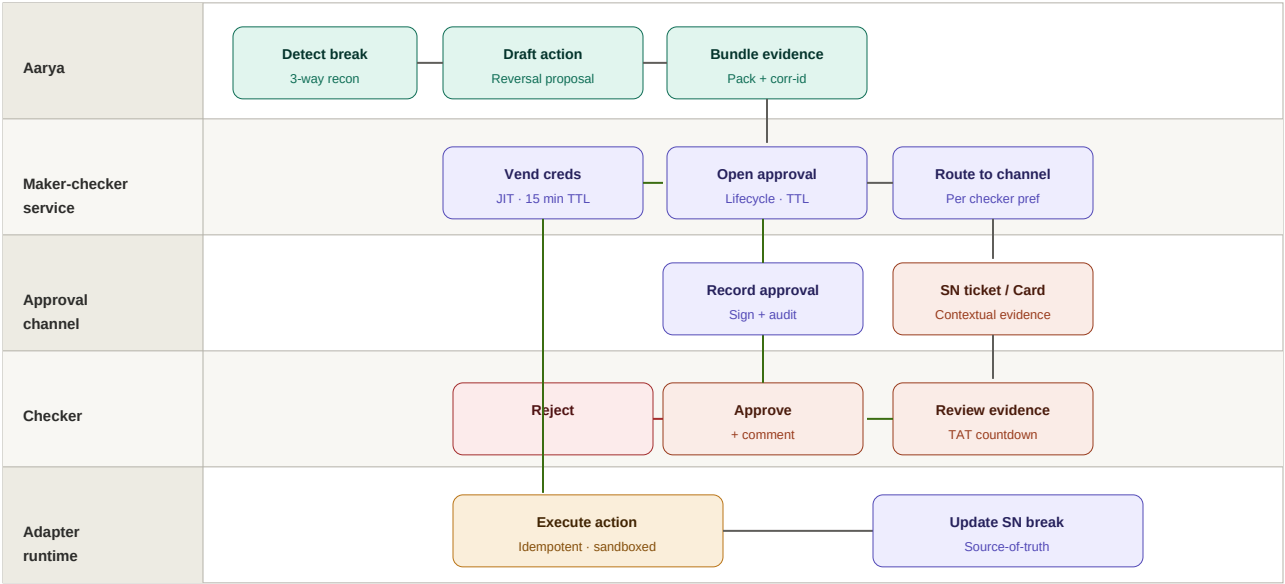
4.4 Determinism & auditability

The engine is rule-based (Rego/OPA), not learned. Every decision is a function of inputs and charter; running the engine again with the same inputs and charter version returns the same tier. The decision record — inputs, charter version, policy IDs evaluated, tier chosen, reason — is persisted to the evidence store before any side effect, even on hard-stop.

5. Maker-checker workflow

When the engine routes to maker-checker (or senior checker), Aakaar opens an approval lifecycle. The worker is the maker: it has prepared the action with full context but not executed it. A named human checker reviews via their preferred channel and approves or rejects.

5.1 Lifecycle



Rejection path: structured reason code returned to maker-checker svc — logged, never auto-applied to production.

Approval channels are pluggable: ServiceNow tickets, Teams adaptive cards, Slack block kits, signed-link email.

Every step in this lane carries the workflow correlation ID; the audit trail can replay the exact decision context.

Diagram — lanes show the maker (Aarya), the maker-checker service, the approval channel, the checker, and the adapter runtime. Approval flows green; rejection flows red back to a structured taxonomy.

5.2 Approval cards must show context, not just consent

Aarya's charter is explicit (12.5): approval cards must include the proposed action, amount, RRN/reference, customer impact, confidence, policy reason, TAT countdown, and deep links to ServiceNow, Salesforce Case, and the recon evidence attachment. Aakaar's maker-checker service composes this card on every approval; the channel adapter renders it natively (ServiceNow rich form, Teams adaptive card, Slack block kit, signed-link email).

5.3 Batch review for low-risk, high-volume categories

For categories the charter explicitly designates as eligible, Aakaar offers **batch review**: the maker-checker service groups proposals, presents aggregate totals, and lets the checker approve all-with-sample (random K), approve all, or drill down. Customer-account-impacting actions are *never* batch-eligible unless the charter declares so.

5.4 Rejection feedback loop

Rejections capture a structured reason from a charter-defined taxonomy (e.g. *wrong category, insufficient evidence, duplicate risk, wrong amount, wrong TAT, threshold breach, customer-impact ambiguity, suspected fraud, portal/network anomaly*). This feeds:

- **Operational queues** — rejected items return to a triage state, optionally reassigned to a human.
- **Governance reporting** — weekly trends, quarterly performance review, replay corpus for prompt updates.
- **Prompt & rule changes** — as inputs to the change-control process, never as automatic production updates. Per Aarya's charter (12.6), no production prompt, threshold, schema mapping, or YAML is updated automatically from rejection feedback.

5.5 Checker availability and TAT

The maker-checker service knows each pool's on-call schedule and live presence. If no approver is available within the TAT-derived deadline, the action escalates one tier (e.g. shift checker → senior checker, then → head of department). If escalation also fails, the action hard-stops with a TAT-breach incident.

6. Quality gates

Three gates sit between the planner and execution. They are independent of the LLM and run on every action, even when the LLM was not involved in producing it.

6.1 Volatility guard

Watches a rolling window of customer-impacting category counts (per Aarya: reversals, refunds, compensations) and emits a **hard-stop signal** when either of two conditions trips:

- Standard-score > 3 against the historical baseline for that category.
- Absolute count exceeds a charter-configured ceiling for that category in the window.

When the guard trips, all open proposals in the affected categories pause; the on-shift officer is alerted; new proposals are rate-limited; release requires a named human incident commander. The guard's state is part of the evidence pack of every subsequent action.

6.2 Velocity check

Prevents duplicate actions on the same logical reference within a window. Keyed on charter-defined identifiers (per Aarya: RRN, transaction reference, customer-account + amount + day). Implemented as a Redis-backed set with TTL = configured window. A duplicate triggers immediate rejection with reason *velocity_dedupe*; the original proposal is the only one allowed.

6.3 Grounding verifier

Aarya's charter (12.2) requires every transaction-specific fact in any narrative she generates to exist in the deterministic evidence pack or approved static reference data. The grounding verifier enforces this:

- Extract claims from the LLM-produced narrative (numbers, dates, identifiers, account references, status statements).
- For each claim, look up a backing record in the evidence pack or the reference catalog.
- If any claim cannot be backed, reject the narrative and demote the action to maker-checker with a *grounding_failed* reason.
- If all claims are backed, attach the claim↔evidence map to the audit trail.

The verifier is itself deterministic (claim extraction is structured, not free-text) and runs in sandboxed compute. It never calls the LLM.

7. Planner & registries

The agentic NL planner from v2 carries forward unchanged in shape, with two additions: it is now **charter-bounded** (the worker's allowlist filters the registry the planner sees), and it consults two further registries beyond capabilities — schemas and prompts.

7.1 Capability registry (unchanged from v2)

Versioned, signed, with parameter schemas, IO types, permissions, and *elicit_prompt* per parameter. Tenants may pin versions; deprecated versions execute with warnings; retired versions are blocked.

7.2 Schema registry

Aarya's charter calls out a **central schema registry** as a v1.2 requirement. This is the catalogue of *data* formats — NPCI UPI files, NBBL Bharat Connect formats, Banking Connect schemas, ServiceNow record shapes, Salesforce object schemas. Each schema is versioned independently of capabilities. Every ingestion event records the schema version it parsed against; schema-drift events are first-class (per Aarya KRI: *all fail closed; no silent mis-parse*).

- Schema definitions are JSON Schema, Avro, or Protobuf depending on the source.
- Each schema has a canonical name (*npci.upi.settlement.v3*), an effective date, a deprecation date, and a sample fixture.
- Adapters declare which schema versions they accept; mismatches hard-stop the affected ingestion.
- The registry exposes a diff API so a new schema version's impact is visible before promotion.

7.3 Prompt registry

Per Aarya's charter (12.2), prompts, narrative templates, tool policies, and redaction instructions are version-controlled, peer-reviewed, and linked to change records. The prompt registry stores all of these as first-class versioned objects.

- Each prompt has a name, version, role (planner / triage / narrative / explanation / redaction), and bound model.
- New versions enter **shadow mode** — run alongside the active version, outputs compared, never executed.
- The **replay/A-B sandbox** runs candidate prompts against a curated historical case set, producing accuracy, hallucination, and grounding metrics.
- Promotion to *active* requires the replay report and an approval record; demotion is a one-click rollback.

7.4 Charter binding at planning time

When the planner is invoked, the worker's charter version, capability allowlist, schema bindings, and prompt bindings are loaded together. The planner sees only what the charter permits; the LLM cannot select a capability the worker isn't allowed to invoke. The registry digest used for the plan is recorded in the audit trail (deterministic replay).

8. Architecture

The framework is layered. The governance and quality-gate layers are new in Aakaar v1; the orchestration, messaging, and data planes are inherited from v2 with refinements.



Every action through Aakaar carries a correlation ID linking charter, plan, evidence, decision, approval, execution, and audit.

Diagram — the Aakaar control fabric, top to bottom. Tenant clients (workers, governance, checkers, tenant systems) talk through the API gateway and identity broker into the governance plane (charter, worker registry, policy, kill-switch). The planning plane reads three registries and a confidence scorer. The quality gates — decision rights engine, anomaly guards, grounding verifier — sit above Temporal. Approval workflows fan out to ServiceNow, Teams, Slack, or email. Bus and state are Kafka + Redis + Postgres + GCS. The data plane runs worker agents in the tenant network. Evidence and audit are content-addressed and hash-chained.

9. Aarya canonical mapping

How each Aakaar component serves Aarya's charter. The diagram makes the boundary between the SaaS control fabric and the in-tenant runtime explicit; the mapping table maps each charter clause to the framework capability that enforces or observes it.

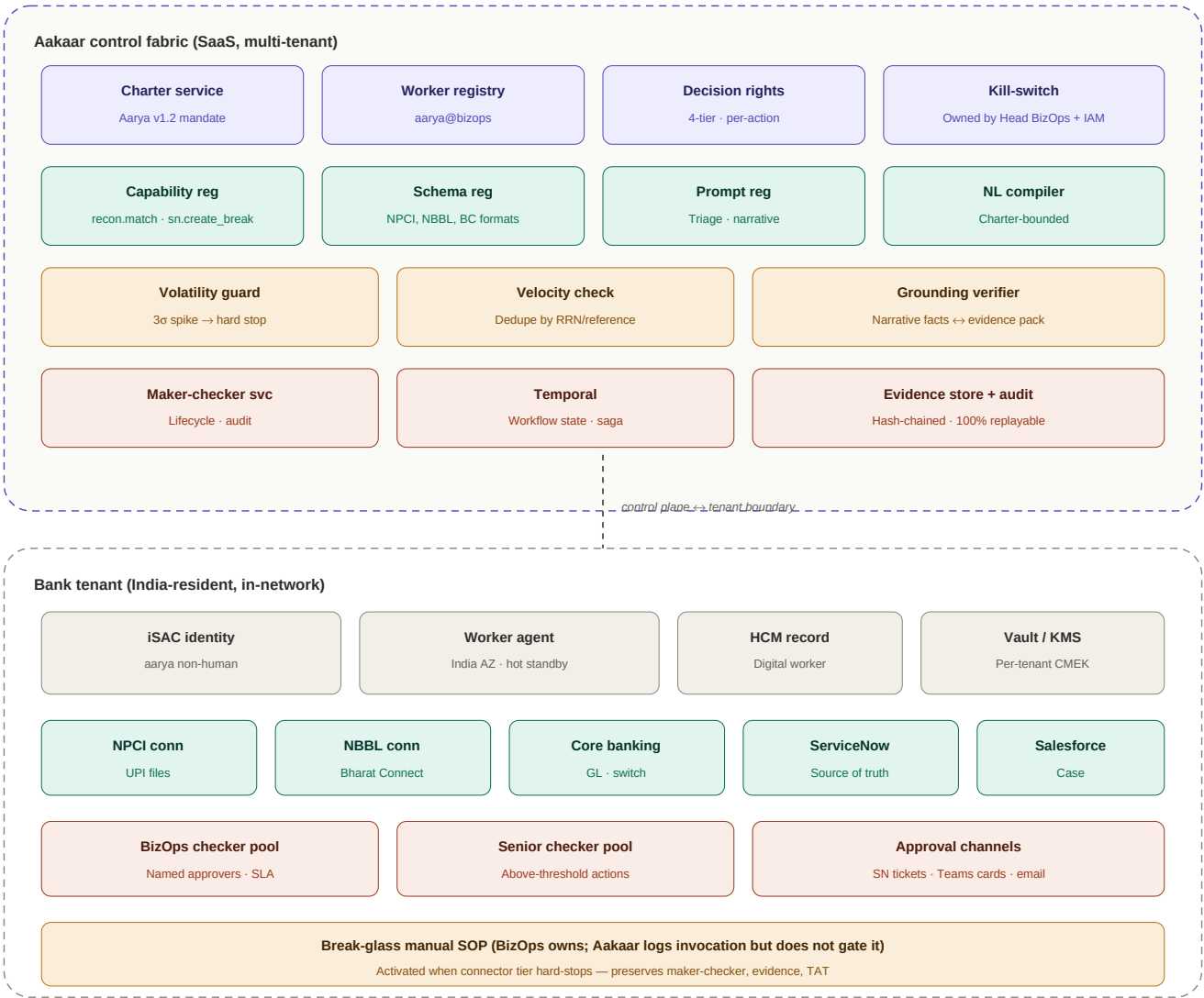


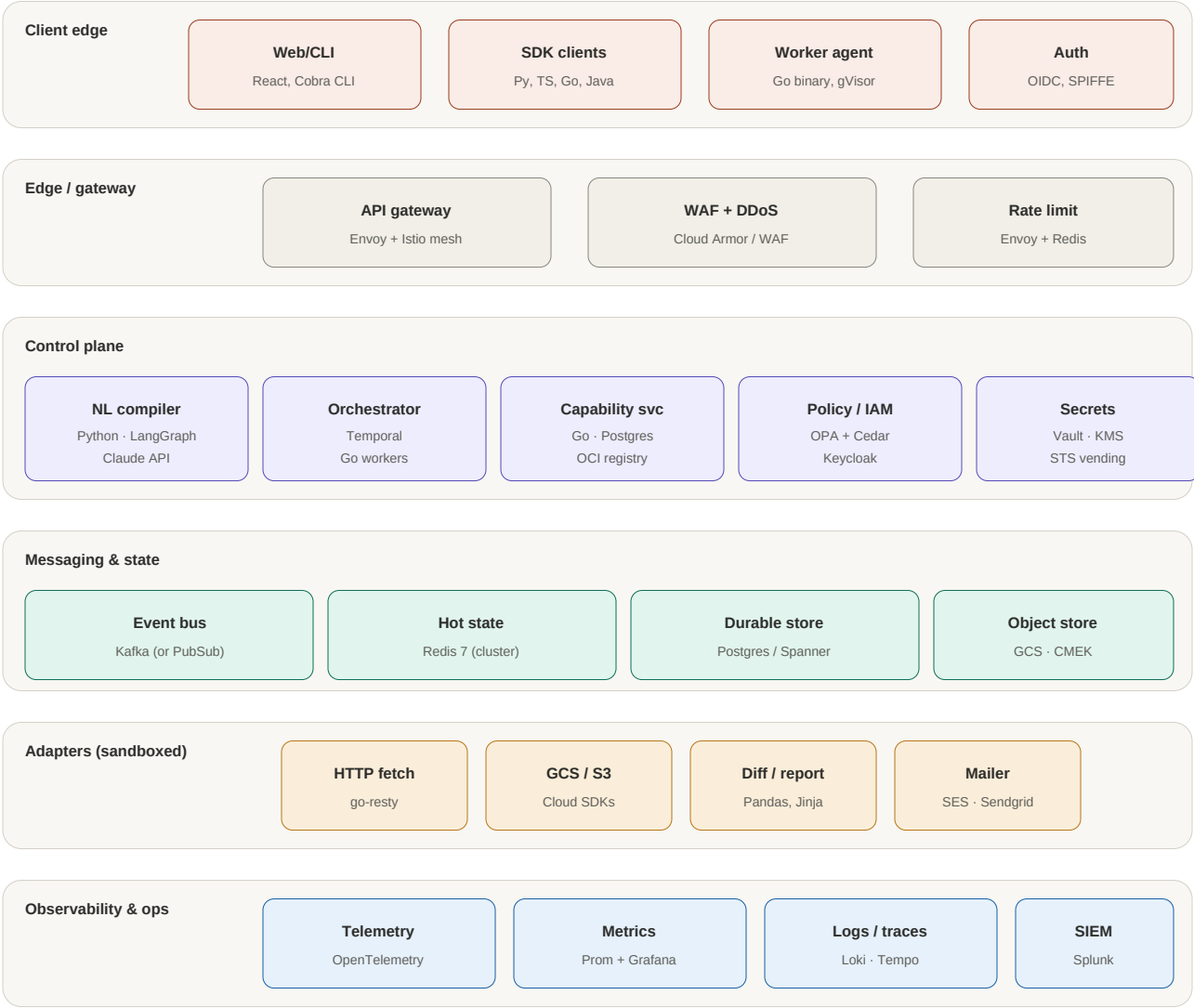
Diagram — the upper region is Aakaar (SaaS, multi-tenant, shared across customers). The lower region is the Bank's tenant deployment (India-resident, in-network). The dashed boundary is where data residency, egress controls, and the maker-checker handoff live.

9.1 Charter-clause-to-component map (selected)

Aarya charter clause	Aakaar component(s) responsible
Section 3 mandate / non-mandate	Charter service stores both lists; Decision Rights Engine evaluates every action; Policy/OPA enforces.
Section 4 decision rights tiers	Decision Rights Engine (4-tier routing); Maker-Checker Service; approval channel adapters.
Section 6 fail-closed on missing evidence/schema	Schema registry; ingestion validators; grounding verifier; evidence store.
Section 6 kill-switch on suspected compromise	Kill-switch service; identity broker (revokes service identities and JIT creds); audit.
Section 9 identity (federated non-human)	Identity broker integrating with iSAC; SPIFFE workload SVID; HCM record link.
Section 9 data residency & minimisation	Per-tenant region pinning; CMEK; tokenisation gate at the LLM gateway; egress allowlists at the agent.
12.1 hot standby + ServiceNow source-of-truth recovery	Active-passive worker pods in second AZ; idempotency cache in Redis; ServiceNow connector reconciles workflow state on restart.
12.1 break-glass manual SOP	Aakaar logs invocation, preserves TAT and evidence linking, pauses automated proposals for the affected flow.
12.2 prompt versioning + A/B replay	Prompt registry; replay sandbox; grounding verifier; promotion approval gate.
12.3 volatility 3σ + velocity check	Anomaly guards (volatility, velocity); rate-limited maker proposals; named-incident-commander release.
12.4 horizontal scaling + TAT-aware queueing	Worker agent fleet (Kubernetes), TAT tracker influences orchestrator priority, graceful degradation rules.
12.5 contextual approval cards + batch review	Maker-checker service composes cards from evidence pack; batch review for charter-allowed categories.
12.6 structured rejection learning, no auto-update	Maker-checker rejection taxonomy; replay sandbox; change-control gate (no automated promotion).
12.6 central schema registry	Schema registry (versioned, India-region, exposed via API to all adapters).
12.7 KRIs (failover, prompt quality, hard-stop, dedupe, latency, reason-code)	Telemetry stack; per-worker scorecard; quarterly evidence pack composer.

10. Technology stack

10.1 Stack overview



CI/CD: GitHub Actions · Argo Rollouts. IaC: Terraform · Helm. Runtime: Kubernetes (GKE/EKS), multi-zone, blue-green deploys.

Diagram — the Aakaar tech stack. Multiple components added vs. v2: decision rights, maker-checker service, schema registry, prompt registry, evidence store. (Diagram simplified for legibility; the architecture diagram on the previous page is the authoritative component list.)

10.2 Message broker decision (carried from v2)

Kafka (or Google Pub/Sub at GCP-first deployments) is the event bus. Per-tenant topics with per-workflow partition keys give strict per-workflow ordering and replayability. Aarya's ingestion of NPCI/NBBL files into the recon pipeline is partitioned by settlement-cycle ID for ordering.

Redis is the hot-state cache. Idempotency keys, rate-limit counters, *velocity* dedupe sets, planner prompt cache. Sub-millisecond reads matter for the velocity check on every action.

Aerospike is not selected. Excellent KV store, but not a broker; using it as one requires re-implementing offsets, consumer groups, and replay on top of records. Parked as a v2 candidate for the idempotency cache only if Redis hits ceiling at scale.

10.3 Orchestration: Temporal

Temporal is the workflow orchestrator. Workflow-as-code gives saga compensation natively (essential for Aarya: a partially-applied refund must be reversible if the GL post fails). Multi-namespace = clean Digital Worker isolation. Replay reconstructs state deterministically — the foundation of the audit-replay KPI.

10.4 New components specific to Aakaar

Component	Choice / approach	Rationale
Charter service	Go service + Postgres + S3-compatible signed-document store	Versioned, signed artifacts; signatures bound to document hashes.
Worker registry	Go service + Postgres	Single source of truth for worker identity, charter binding, status.
Decision Rights Engine	OPA (Rego) policies + Go evaluator	Deterministic, auditable; co-located with the planner for low latency.
Maker-Checker Service	Go service + Postgres (lifecycle) + adapter SDK	Channel adapters for ServiceNow, Teams, Slack, email; pluggable for new channels.
Anomaly guards	Go service + Redis (sliding windows) + ClickHouse (history)	Volatility uses ClickHouse for fast historical statistics; velocity uses Redis sets with TTL.
Grounding verifier	Python service; rule-based claim extraction + lookup against evidence pack	Deterministic; no LLM dependency; isolated from the planner.
Schema registry	Confluent Schema Registry (or self-hosted Apicurio) for Avro/Protobuf; JSON Schema in custom Go service	Battle-tested for streaming schemas; custom service for non-streaming formats.
Prompt registry	Go service + Postgres + LangSmith-style trace store	Versioning, A/B replay, peer review hooks.
Evidence store	S3/GCS with object lock + content-addressed (sha256) keys + Postgres index	Immutable, append-only, signed; ties to audit trail.
Identity broker	Keycloak (OIDC) + SPIFFE/SPIRE for workload identity + custom JIT-elevation service	Federates with iSAC and similar tenant IdPs; SPIFFE for workload SVIDs; JIT vending under audit.
TAT tracker	Go service + Redis (live deadlines) + Postgres (per-cycle history)	Drives orchestrator priority; live to settlement cycles per tenant.

Component	Choice / approach	Rationale
Kill-switch	Control-plane service with single API; cascades to identity broker, orchestrator, agent	RTO $\leq$ 60 s; revokes SVIDs, drains queues, freezes orchestrator.

10.5 Inherited choices (unchanged from v2)

- **LLM provider:** Anthropic Claude via the platform LLM gateway; Bedrock available for AWS-resident tenants.
- **Planner runtime:** Python + LangGraph (state-machine framework for bounded agentic loops).
- **Worker agent:** Single Go binary; runs on Linux at the tenant.
- **Sandbox:** gVisor default; Firecracker microVMs for high-isolation tier.
- **Policy:** OPA (Rego) + Cedar where ABAC concision matters.
- **Secrets / KMS:** HashiCorp Vault + cloud KMS; per-tenant CMEK.
- **Observability:** OpenTelemetry → Prom + Loki + Tempo + Grafana; SIEM (Splunk) integration.
- **IaC:** Terraform + Helm; Argo Rollouts for progressive deployment.

11. Non-functional requirements & SLAs

11.1 Service level objectives

Public Enterprise-tier SLAs. Standard tier loosens availability by one nine and doubles latency budgets.

Metric	Enterprise SLA	Internal SLO	Notes
Control plane availability	99.9%	99.95%	Gateway + governance + planner
Data plane availability	99.95%	99.99%	Bus + orchestrator + agents
NL plan p95 latency	< 5 s	< 4 s	Excludes user clarification turns
Decision rights eval p95	< 50 ms	< 20 ms	Per action
Anomaly guard p95	< 30 ms	< 15 ms	Per action
Grounding verifier p95	< 200 ms	< 150 ms	Per narrative
Step dispatch p95	< 500 ms	< 300 ms	From task ready to agent receive
Maker-checker open-to-deliver p95	< 30 s	< 15 s	From ready-to-approve to checker notified
Kill-switch RTO	≤ 60 s	≤ 30 s	End-to-end revoke + drain
Workflow durability	99.999%	99.9999%	No lost workflows after acknowledgment
RPO (data loss)	≤ 60 s	≤ 30 s	Synchronous replication
DR RTO (region failover)	≤ 15 min	≤ 10 min	Warm-standby
Audit replay (sampled)	100%	100%	Hard requirement; quarterly drill
Cross-tenant isolation incidents	0	0	Hard requirement

11.2 Aarya-specific operational targets

Inherited from charter section 12.7. Aakaar must produce evidence for these on the per-worker scorecard:

- Quarterly DR drill within approved RTO/RPO — Aakaar exposes the drill harness.
- Zero critical hallucination defects per prompt release — gated by replay sandbox + grounding verifier.
- 100% review of volatility hard-stop events with root cause — Aakaar produces the incident pack.
- Median checker approval latency within agreed SLA; critical TAT cases escalated before 90% TAT consumption.
- ≥ 95% of rejected proposals carry a structured reason code — enforced by the rejection UI.

11.3 Security requirements

- **NFR-S1.** Zero-trust east-west: mTLS via service mesh; SPIFFE workload identity for every component.
- **NFR-S2.** Adapters run in sandboxed execution (gVisor minimum, Firecracker for high-isolation tier).
- **NFR-S3.** All adapter and prompt artifacts are signed (Sigstore) with SLSA L3 provenance.
- **NFR-S4.** Per-tenant CMEK with quarterly rotation; secrets vended JIT (TTL ≤ 15 min).
- **NFR-S5.** Egress allowlists per capability; default-deny on agent network egress.
- **NFR-S6.** Append-only audit log to write-only sink; tamper-evident (hash-chained); replayable.

- **NFR-S7.** Prompt-injection defenses: instruction isolation, structured output, registry-restricted output, user content never treated as instructions.
- **NFR-S8.** Data residency enforced at planning, ingestion, storage, and inference (Aarya: India-only).
- **NFR-S9.** PII tokenisation at the LLM gateway; raw payment/customer data never leaves the tenant region.
- **NFR-S10.** Compliance posture: SOC 2 Type II, ISO 27001, GDPR; HIPAA-ready BAA at Enterprise; sector-specific (RBI guidelines) at India deployments.

11.4 Scalability targets

- 10,000 active Digital Workers per region across all tenants.
- 100,000 concurrent in-flight workflows per region.
- 1M workflow steps/hour aggregate per region.
- 50,000 connected worker agents per region (long-poll).
- Per worker: charter-defined throughput (Aarya: capable of 1M+ recon records per settlement cycle).

12. Client-side requirements

Aakaar is SaaS. The customer is responsible for: (a) running the worker agent in their network, (b) federating identity, (c) providing tenant system credentials via Vault dynamic secrets or BYOK, and (d) maintaining checker pools. Everything else is operated by Aakaar.

12.1 Worker agent host

Requirement	Specification	Notes
OS	Linux (kernel ≥ 5.4)	Windows in v1.5
CPU	≥ 2 vCPU (4 vCPU recommended)	Per agent; scale-out for parallelism
Memory	≥ 4 GB (8 GB recommended)	Sandbox overhead included
Disk	≥ 20 GB free, SSD strongly preferred	Adapter image cache + temp dirs
Container runtime	containerd, Docker, or Podman	Sandboxing via gVisor or microVM
Time sync	NTP within ± 5 s of UTC	Required for credential validity + idempotency keys
DNS	Egress DNS (public or split-horizon)	For bus + registry reachability
Hypervisor	KVM extensions enabled (high-isolation tier)	For Firecracker microVMs

12.2 Network

- **Outbound only to Aakaar.** Agent makes outbound TLS (443) to *bus.aakaar.io*, *registry.aakaar.io*, *identity.aakaar.io*. No inbound holes required.
- **Tenant-internal egress** is governed by the worker's charter-bound egress allowlist (e.g. Aarya: NPCI endpoints, NBBL endpoints, internal Banking Connect bus, ServiceNow, Salesforce).
- **Proxy support:** HTTP/HTTPS with auth (basic, NTLM SPN), and SOCKS5.
- **Bandwidth:** 1 Mbps sustained per active workflow; bursts to 100 Mbps for large file transfers (settlement files).

12.3 Identity & auth

- **Worker bootstrap.** One-time enrollment token printed during agent install, exchanged for SPIFFE workload SVID. Subsequent auth is mTLS.
- **User auth.** OIDC against the tenant's IdP (Aarya: iSAC). SCIM 2.0 for provisioning charter owners and checker pool members.
- **Federated non-human identity.** The Digital Worker has a record in the tenant HCM (per Aarya). Aakaar binds its operational identity to that HCM record — lifecycle changes (worker decommission) propagate.
- **JIT elevation.** Each tier-routed action vends a short-lived credential scoped to the action's permission requirements. Audit records the elevation.

12.4 Tenant system integrations (per worker)

- **State-of-record system.** Each charter names one (Aarya: ServiceNow *u_bizops_recon_break*). Aakaar reconciles workflow state with this on agent restart.
- **Approval channels.** ServiceNow workflows, Teams adaptive cards, Slack block kits, signed-link email — one or more per worker per tier.

- **Source connectors.** NPCI, NBBL, Banking Connect for Aarya. Each connector ships as a signed adapter image registered in the capability registry.
- **Notification distribution lists.** Allowlisted at the maker-checker service; charter governs who is on each list.

12.5 SDKs

First-class SDKs in Python, TypeScript, Go, Java. Operations: *workers.list*, *workers.invoke(intent)*, *workers.plan(intent)* (returns DAG without executing), *workers.execute(plan)*, *workers.watch(workflow_id)* (SSE), *workers.cancel*, *workers.suspend* (kill-switch), *capabilities.list*, *schemas.list*. SDKs handle auth refresh, retries, idempotency keys, and correlation ID propagation.

12.6 Onboarding sequence (Aarya as example)

- Bank's tenant is provisioned in Aakaar; CMEK key is created in India region.
- iSAC federation is configured; SPIFFE trust domain extended to the tenant.
- Aarya's charter v1.2 is uploaded; reviewers route through Aakaar; signatories sign.
- Worker registry creates *aarya@bizops*; HCM record is created in the Bank's HCM via SCIM.
- Capability allowlist, schema bindings, prompt bindings are bound; checker pools are imported.
- Worker agents are installed in the Bank's in-network deployment with hot-standby in a second AZ.
- Credentials for NPCI, NBBL, ServiceNow, Salesforce are vaulted; egress allowlists configured.
- DR drill rehearsed; kill-switch drilled; first settlement-cycle run is a shadow comparison against legacy SOPs.
- Activation: charter goes live; Aarya enters production with a named day-one incident commander.

13. Algorithms (pseudocode)

Contracts between framework components. Language-agnostic; no implementation detail. Core flows from v2 (capability data structure, agentic planning loop, DAG validation, version resolution, idempotency, retry classification, tenant-partitioned dispatch) carry forward unchanged. The new flows below cover charter binding, decision rights evaluation, anomaly guards, grounding verification, and maker-checker lifecycle.

13.1 Digital Worker invocation context

```
WorkerContext {
  worker_id : uuid // e.g. aarya@bizops
  charter_version : string // e.g. "v1.2"
  capability_allow : list<capability_pattern>
  schema_bindings : map<source, schema_version>
  prompt_bindings : map<role, prompt_version>
  checker_pools : map<tier, pool_id>
  tier_thresholds : map<category, threshold_set>
  hard_stops : list<category>
  jit_scope_tmpl : map<capability, scope_template>
  status : enum {active, suspended, decommissioned}
}
```

13.2 Decision rights evaluation

```
function evaluate_decision_rights(action, ctx: WorkerContext) -> Tier:
  if ctx.status != active: return HARD_STOP(reason="worker_suspended")
  if kill_switch.active(ctx.worker_id): return HARD_STOP(reason="kill_switch")

  // 1. Mandate check
  if action.category in ctx.hard_stops: return HARD_STOP(reason="out_of_mandate")
  if not allowlist_match(action.capability, ctx.capability_allow):
    return HARD_STOP(reason="not_in_allowlist")

  // 2. Schema integrity
  if action.requires_schema and not schema_registry.valid(action.schema):
    return HARD_STOP(reason="schema_drift")

  // 3. Anomaly gates
  if volatility_guard.tripped(action.category, ctx):
    return HARD_STOP(reason="volatility_3sigma")
  if velocity_check.duplicate(action.dedupe_key, ctx):
    return HARD_STOP(reason="velocity_dedupe")

  // 4. Confidence + thresholds
  t := ctx.tier_thresholds.get(action.category)
  if action.confidence < t.min_confidence:
    return HARD_STOP(reason="low_confidence")

  // 5. Tier selection
  if action.amount > t.senior_threshold or action.cross_system:
    return SENIOR_CHECKER(pool = ctx.checker_pools.senior)
  if action.customer_impacting or not action.in_auto_allowlist:
    return MAKER_CHECKER(pool = ctx.checker_pools.maker)
  return AUTO_EXECUTE
```

13.3 Volatility guard

```
function volatility_guard(category, ctx) -> bool:
  baseline := clickhouse.stats(category, window=ctx.baseline_window)
  current := redis.count(category, window=ctx.live_window)

  z_score := (current - baseline.mean) / baseline.stddev
  abs_cap := ctx.tier_thresholds.get(category).abs_ceiling

  if z_score > 3.0 or current > abs_cap:
    state.set_tripped(category, evidence={z_score, current, baseline})
    notify(on_shift_officer, ctx)
    return true
  return false

// Release requires named incident commander, not automatic decay
function volatility_release(category, commander, justification):
  require auth(commander, role=incident_commander)
  state.clear(category, by=commander, reason=justification)
  audit.write(category, commander, justification)
```

13.4 Velocity check

```
function velocity_check(dedupe_key, ctx) -> bool:
  // Window keyed on charter-defined fields (Aarya: RRN + tx_ref)
  window_key := "vel:" + ctx.worker_id + ":" + dedupe_key
  ttl := ctx.tier_thresholds.window_seconds

  // SETNX: returns true if absent (not duplicate), false if present (duplicate)
  is_new := redis.setnx(window_key, current_workflow_id, ttl)
  return not is_new // true means duplicate
```

13.5 Grounding verifier

```
function verify_grounding(narrative, evidence_pack) -> GroundingResult:
  claims := extract_claims(narrative) // structured: numbers, ids, dates
  unbacked := []

  for claim in claims:
    match claim.kind:
      case AMOUNT : ok := evidence_pack.has_amount(claim.value)
      case DATE : ok := evidence_pack.has_date(claim.value)
      case TXN_REF : ok := evidence_pack.has_ref(claim.value)
      case ACCOUNT : ok := evidence_pack.has_account(claim.value)
      case STATUS : ok := static_refdata.has_status(claim.value)
      default : ok := false
    if not ok: unbacked.append(claim)

  return GroundingResult(
    passed = unbacked.empty,
    mapping = claim_to_evidence_map(claims),
    unbacked = unbacked,
    reason = "grounding_failed" if unbacked else null
  )
```

13.6 Maker-checker lifecycle

```
function open_approval(action, evidence_pack, tier, ctx) -> ApprovalId:
  pool := ctx.checker_pools.get(tier)
  tat := tat_tracker.deadline(action)
  card := compose_card(action, evidence_pack, tier, tat)
  ttl := tat - safety_buffer

  approval := approvals.create(
    tier = tier,
    pool = pool,
    card = card,
    ttl = ttl,
    ctx = ctx,
    evidence_id = evidence_pack.id
  )
  for channel in pool.channels:
    channel.deliver(approval) // SN, Teams, Slack, email
  return approval.id

function on_decision(approval_id, decision, checker, reason_code?, comment?):
  require auth(checker, in_pool=approval.pool)
  require fresh_evidence(approval.evidence_id)

  match decision:
    case APPROVE:
      creds := identity_broker.vend(approval.action.scope, ttl=15min)
      dispatch(approval.action, creds)
      audit.write(approval, APPROVED, checker, comment)
    case REJECT:
      require reason_code in REJECTION_TAXONOMY
      state.cancel(approval, reason_code, comment)
      feedback.record(reason_code, action.category, ctx.worker_id)
      audit.write(approval, REJECTED, checker, reason_code, comment)

function on_ttl_expiry(approval_id):
  if approval.tier < SENIOR_CHECKER:
    escalate_one_tier(approval)
  else:
    hard_stop(approval, reason="tat_exhausted")
```

13.7 Kill-switch cascade

```
function kill_switch(worker_id, owner) -> KillResult:
  require auth(owner, in=ctx.kill_switch_owners)
  t0 := now()

  parallel:
    worker_registry.set_status(worker_id, suspended)
    identity_broker.revoke_svid(worker_id)
    identity_broker.revoke_jit_creds(worker_id)
    orchestrator.freeze(worker_id) // pauses all in-flight workflows
    maker_checker.cancel_open(worker_id, reason="kill_switch")
    agents.send_drain(worker_id)
  await all

  audit.write_kill_event(worker_id, owner, t0, now())
  notify(charter_owners, security_team, ctx.escalation_tree)
  return KillResult(rto = now() - t0)
```

14. Risks & open questions

Risk	Likelihood	Mitigation
Decision rights engine misclassifies an action and waves a sensitive one through.	Medium	Default-deny; mandatory schema for capability declarations including <i>customer_impacting</i> flag; quarterly red-team replay; rollout shadow-mode-first per worker.
Volatility guard false positives during legitimate spikes (e.g. festival days).	High	Calendar-aware baselines; per-category absolute caps tunable at the charter level; named incident-commander release path.
Grounding verifier fails to catch hallucinations in narrative.	Medium	Verifier handles structured facts only; narratives whose claim graph is incomplete demote action to maker-checker (never auto-execute) regardless.
LLM cost dominates margin as workers scale.	High	Per-worker token budgets; planner cache; smaller model on warm cache; embedding-precomputation; capability retrieval narrows context.
Charter ambiguity — clauses that are ambiguous when machine-evaluated.	High	Structured charter authoring tool with linting; required machine-evaluable clauses for tier rules; reviewer signs only after lint pass.
Approval channel outage (Teams down) blocks all maker-checker.	Medium	Multi-channel routing; if all configured channels fail, maker-checker enters degraded mode (signed-link email + SMS).
Kill-switch exercised by mistake.	Low	Two-person rule for kill-switch (charter owner + IAM); easily-reversible; resume operation requires the same two-person re-attestation.
State-of-record drift between Aakaar and tenant system (ServiceNow).	Medium	Aakaar reconciles on agent restart and on every adapter retry; ServiceNow remains source-of-truth for break lifecycle (per Aarya 12.1).
Schema-registry version skew between adapter and source.	Medium	Adapters declare accepted ranges; mismatches hard-stop; deprecation windows announced 90d in advance.
Worker agent compromise pivots to tenant network.	Low-Medium	Sandbox; outbound-only; egress allowlists; vended creds with TTL; agent runs unprivileged; SLSA-verified images.

Open questions for v1 review

- Should Aakaar offer a hosted charter authoring tool, or rely on the tenant's document tooling and import signed PDFs?
- Default sandbox: gVisor for everyone, or Firecracker for charters that touch payment data (Aarya tier)?
- Should batch-review eligibility be a charter-level setting, or per-action-category configuration outside the charter?
- How do we surface charter version drift to checkers (banner on every approval card? quarterly digest?).
- Cross-charter changes — if a capability version is retired, do we auto-amend every charter that pinned to it, or hard-stop those workers until reviewed?
- Does the planner share registry context across workers in the same tenant, or is every worker fully isolated for context?